Primo lavoro su GitHub

2INF · lezione del 25/09/2026

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

Che cosa abbiamo fatto

1. **Quando:** 25/09/2026, 2 ore, Informatica.
2. **Argomenti:** Creazione dell’account Git; primo lavoro su GitHub: repository, file, commit e versioni.

La teoria e l’esercitazione

Dispensa · Classe 2 · Informatica · v2.2

GitHub e il versioning

quello che abbiamo scritto insieme alla lavagna, spiegato per bene

Partiamo dallo schema della **nostra lavagna** e lo apriamo pezzo per pezzo: cos’è GitHub, che tipo di servizio è (PaaS, Freemium) e come funzionano le **versioni**. Con esempi e senza comandi da digitare. Classe 2 · Informatica.

1. Partiamo dalla nostra lavagna

Ecco, ridisegnato in ordine, lo schema che abbiamo costruito insieme alla lavagna. Ogni riquadro è una tappa: nelle prossime pagine le apriamo una per una.

GitHub

è un’AZIENDA è un SERVIZIO

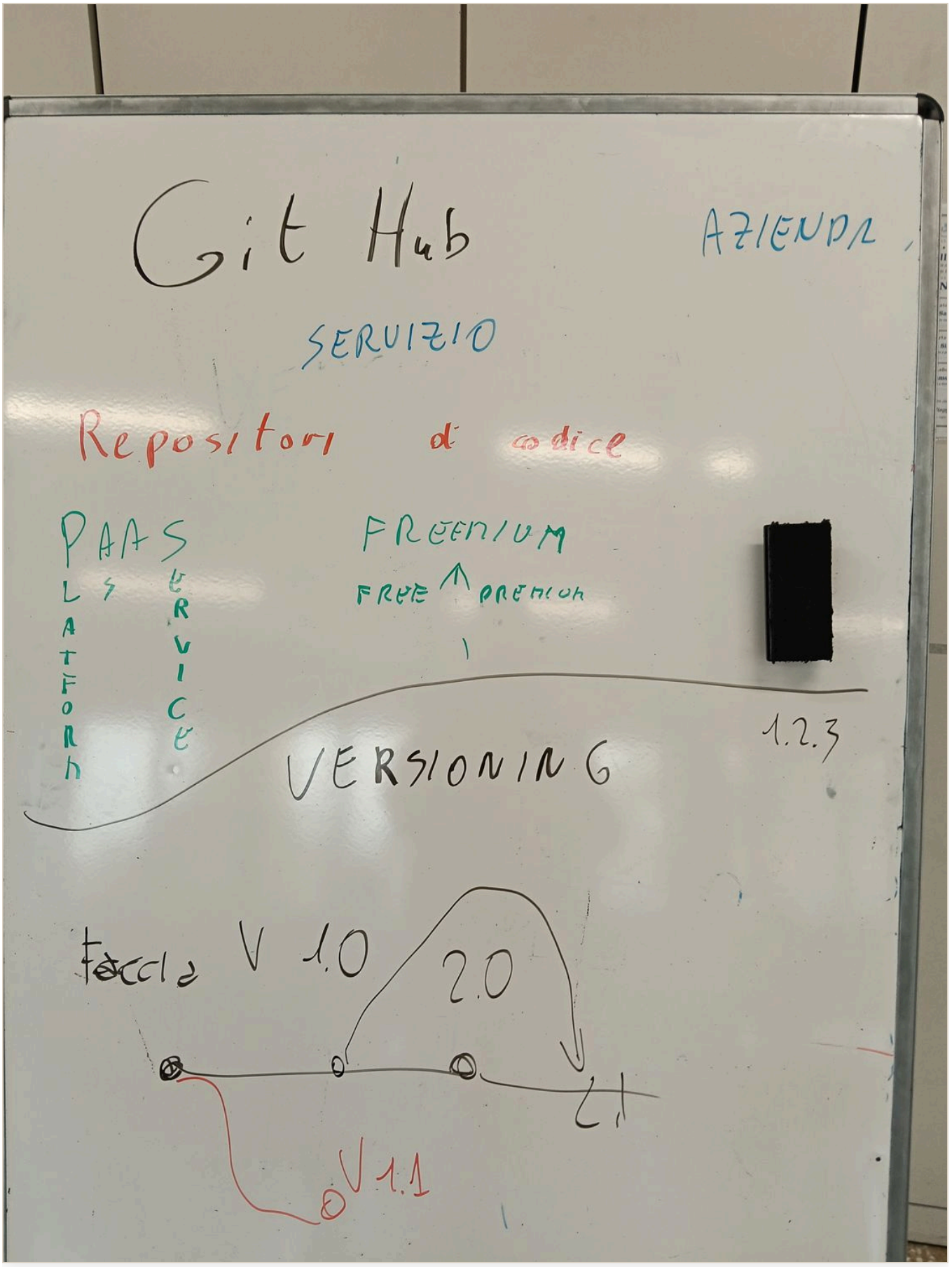
tiene Repository di codice

PaaSPlatform as a Service FREEMIUMfree premium

VERSIONING le versioni: 1 . 2 . 3

v1.0v1.1v2.0 le versioni pubblicate crescono nel tempo

Lo schema della lezione: GitHub è azienda e servizio; tiene repository di codice; come servizio è una piattaforma (PaaS) col modello Freemium; e il progetto cresce per versioni (versioning).



La **lavagna della lezione** (3ª ora): lo schema originale scritto in classe, da cui parte questa dispensa.

2. GitHub: un’azienda e un servizio

Alla lavagna abbiamo scritto due parole sopra “GitHub”: **azienda** e **servizio**. Vediamo perché tutte e due sono vere.

È un’azienda

Dietro GitHub c’è una **società vera** — **GitHub, Inc.**, nata nel 2008 e dal 2018 di proprietà di **Microsoft**. Come ogni azienda deve mantenersi: ci arriviamo al punto “Freemium”.

È un servizio (online)

GitHub è un **sito** dove metti i tuoi progetti **online**: per tenerli al sicuro, mostrarli e lavorarci in gruppo. Non è un programma che installi col CD: lo usi dal **browser**.

Attenzione a non confondere: Git è lo **strumento** (la tecnologia che tiene la storia di un progetto, gratis e di nessuno); **GitHub** è il **servizio online** (l’azienda) dove quei progetti vivono in rete. Git è il **motore**, GitHub è il **garage online**.

3. Che cosa tiene: i repository di codice

Sulla lavagna, sotto “servizio”, abbiamo scritto **“repository di codice”**: è ciò che GitHub custodisce.

Un **repository** (in breve “repo”) è la **cartella-progetto** con dentro i file **e tutta la loro storia**. Ogni salvataggio della storia si chiama **commit** (una “fotografia” del progetto con un messaggio). Se lavori in gruppo, puoi aprire un **branch** (una strada di lavoro parallela) e poi riunirlo con un **merge**.

Esempio. Il tuo gioco “SpaceRun” è un repository su GitHub: dentro c’è il gioco **e** il diario di tutte le modifiche. Puoi tornare a com’era la settimana scorsa in qualsiasi momento.

4. Che tipo di servizio è: PaaS

Alla lavagna: **PaaS = Platform as a Service**, cioè “piattaforma **come servizio**”. Vuol dire che qualcuno ti mette a disposizione online una **piattaforma già pronta**, e tu la usi senza costruirti e mantenere i computer, i server e tutto il resto.

Analogia della cucina. Per aprire un ristorante puoi **costruirti la cucina da zero** (tanta fatica) oppure **affittare una cucina già attrezzata** e pensare solo a cucinare. GitHub è la cucina già pronta: tu porti il progetto, il resto lo mette lui.

La famiglia “…aaS”

PaaS non è da solo: fa parte di una famiglia di servizi online, tutti con la sigla che finisce in **“aaS”** (*as a Service*). Cambia quanto lavoro fai tu:

Sigla	Nome	Cosa ti danno pronto	Esempio
IaaS	Infrastructure as a Service	Solo i “muri e la corrente”: server nudi.	Amazon AWS, Google Cloud
PaaS	Platform as a Service	La cucina già attrezzata: piattaforma pronta.	GitHub, Netlify
SaaS	Software as a Service	Il piatto già servito: un programma pronto nel browser.	Gmail, Google Documenti

Come ricordarli — la pizzeria. IaaS = affitti il locale vuoto. **PaaS** = affitti la cucina attrezzata. **SaaS** = ti fai consegnare la pizza pronta. Più scendi, meno fatica fai.

5. Come si paga: il modello Freemium

Sulla lavagna: **Freemium = FREE PREMIUM**. È una parola nata unendo **free** (gratis) e **premium** (a pagamento). Significa: le funzioni **di base sono gratis** per tutti; se vuoi funzioni **in più**, paghi un abbonamento.

Esempio che conosci. Tanti giochi per telefono e app come Spotify funzionano così: gratis con qualche limite, a pagamento per togliere pubblicità o avere funzioni extra. **GitHub è uguale:** a te studente basta e avanza la parte gratis.

Ecco perché “azienda” e “Freemium” stanno insieme: GitHub è un’azienda, quindi deve guadagnare; lo fa lasciando gratis l’uso normale e facendo pagare le funzioni avanzate a chi ne ha bisogno (aziende, team grandi).

6. Il versioning: le versioni 1.2.3

L’ultima parte della lavagna: **VERSIONING**, con il numero 1.2.3. Quando un progetto raggiunge un punto “buono”, gli si dà un **numero di versione** fatto di tre cifre:

Cifra	Nome	Aumenta quando…
1	major	cambi **tutto**, in modo grosso (magari incompatibile col vecchio)
2	minor	aggiungi una **funzione nuova**, ma il resto funziona ancora
3	patch	correggi solo un **bug**, niente di nuovo

Esempio. v1.0.0 prima versione giocabile ⟶ aggiungi i suoni: v1.1.0 ⟶ correggi un bug: v1.1.1 ⟶ rifai tutto da capo: v2.0.0.

Il grafo che abbiamo disegnato

Alla lavagna avevamo disegnato la linea delle versioni con un ramo che si stacca e rientra. Ecco cosa racconta: il progetto va avanti su main, ogni tanto si pubblica una versione (le bandierine), e un ramo di lavoro si può staccare e poi riunire.

v1.0v2.0

v1.1 (ramo)

Da v1.0 il progetto cresce; un ramo porta la v1.1 e si riunisce; poi un grande cambiamento porta alla v2.0.

Regola d’oro: una versione già pubblicata **non si riusa mai**. Se cambi anche una sola cosa, prima **aumenti** il numero e poi pubblichi.

7. Casi d’uso: 15 grafi con spiegazione (fino a 5 livelli)

Vediamo Git/versioning “in azione”. In ogni grafo il **pallino** è un commit/versione e la **linea** è il tempo (da sinistra a destra). I primi sono semplici; gli ultimi arrivano a **cinque livelli** di rami.

Come si legge un grafo: ogni **pallino** è un **commit** (una fotografia salvata). La **linea orizzontale** in alto è di solito main, il ramo principale. Le linee colorate che si staccano sono **branch** (rami di lavoro). Quando due linee si riuniscono è un **merge**. Le **bandierine gialle** sono le **release** (versioni pubblicate, es. v1.0).

1. Sviluppo lineare (da soli, tutto dritto)

Il caso più semplice: un solo autore, nessun ramo. Ogni commit segue il precedente in fila. La storia è una linea dritta. Va benissimo per progetti piccoli.

avviopaginastiletestifixfine

2. Una funzione su un branch, poi merge

Provi una funzione nuova su un ramo separato, così il main resta sempre funzionante. Quando è pronta, la **unisci** (merge) al main.

main

funzione “login” merge

3. Due funzioni in parallelo, merge in tempi diversi

Due rami partono insieme ma finiscono in momenti diversi: uno si fonde prima, l’altro dopo. Il main raccoglie i pezzi man mano che sono pronti.

main

grafica

suoni (finisce dopo)

4. Hotfix urgente durante una funzione lunga

Stai lavorando a una funzione che richiede giorni, ma esce un **bug grave** in produzione. Apri un ramo **hotfix** corto, lo correggi e lo unisci subito; poi torni alla tua funzione.

main

hotfix (bug)

la mia funzione lunga (continua)

5. Le release pubblicate (bandierine sul main)

Man mano che il progetto avanza, si **pubblicano** le versioni: v1.0 giocabile, poi correzioni e aggiunte fino a un cambiamento grosso v2.0. Ogni bandierina è una versione “congelata”.

v1.0v1.1v1.2v1.3v2.0 +suoni+livellifixtutto nuovo

6. Un ramo abbandonato (idea scartata)

Non tutte le idee funzionano. Qui un ramo viene provato ma **mai unito**: si decide che non va e lo si lascia lì. Il bello di Git: il main non ne risente per niente.

main

idea “grafica 3D” — abbandonata

7. Un ramo che nasce da un altro ramo

A volte una funzione è così grande che, mentre la sviluppi, ne stacchi un pezzo su un **sotto-ramo**. Prima rientra il sotto-ramo nel ramo, poi il ramo nel main. Rami dentro rami.

main

funzione “mappa”

sotto-ramo “zoom”

8. Grande integrazione (progetto di gruppo)

Nel lavoro di squadra ognuno ha il suo ramo (grafica, movimenti, audio, menu). Alla fine confluiscono tutti nel main con le loro Pull Request: nasce la versione completa.

main grafica movimenti audio menu v1.0

9. Ramo di manutenzione a lungo termine

Esci con la v1.0 e il main va avanti verso la v2.0 (tutto nuovo). Ma chi usa ancora la v1 ha bisogno di correzioni: si tiene un **ramo di manutenzione** parallelo dove escono v1.1, v1.2 solo con i fix.

main v2.0 v1.0

ramo manutenzione v1.x v1.1v1.2

10. Revert: annullare una modifica sbagliata

Un commit ha introdotto un problema. Invece di cancellare la storia (non si fa), si aggiunge un **nuovo commit** che **annulla** quello sbagliato: è il **revert**. La storia resta onesta e il progetto torna a posto.

main commit sbagliato revert (annulla)

11. Conflitto di merge risolto

Due rami hanno cambiato **la stessa riga** in modo diverso. Quando si uniscono, Git non sa quale tenere: è un **conflitto**. Una persona decide cosa tenere e chiude il merge con un commit speciale di “risoluzione”.

main cambio A cambio B (stessa riga)

merge risolto

12. Squadra a 5 rami con due release

Un progetto vero di gruppo: sul **main** si pubblicano le versioni, mentre **quattro rami** diversi (grafica, movimenti, audio, menu) lavorano in parallelo e rientrano in momenti diversi. Cinque linee in tutto.

main

v1.0v2.0

grafica

movimenti

audio

menu

13. Rami annidati: cinque livelli di profondità

Una funzione grande si divide in pezzi: dal main nasce un ramo, da quel ramo un **sotto-ramo**, e ancora più giù. Ogni livello rientra nel precedente. Qui si scende di **cinque livelli**.

main livello 1 livello 2 livello 3 livello 4

14. Due versioni major mantenute insieme

Dopo la v1.0 il main punta alla v2.0, ma la v1 resta in uso: un **ramo di manutenzione** sforna v1.1/v1.2 (solo fix), e più tardi anche la v2 avrà il suo ramo di manutenzione. Tante linee vive insieme.

main v1.0v2.0

v1.1v1.2v1.3ramo manutenzione v1.x

v2.1ramo manutenzione v2.x

15. Storia realistica: rami, hotfix, revert e conflitto

Il quadro completo: il main avanza con le release; una **feature** lunga con un **sotto-ramo**; un **hotfix** urgente; un **revert** per annullare un errore; e un **merge con conflitto** risolto. Cinque livelli di attività insieme.

main errorerevert v2.0

hotfix

feature sotto-ramo

ramo parallelo (stessa riga) ⟶ merge con conflitto

In sintesi: con pochi elementi — commit, branch, merge, tag/release — si descrivono **tutte** le situazioni di un progetto reale, dalla più semplice (linea dritta) alle più complesse (integrazione di squadra, manutenzione parallela, conflitti). Saper **leggere** questi grafi è già metà del lavoro.

8. Riepilogo (la lavagna in tre righe)

1. **GitHub** è un'**azienda** e un **servizio** online dove tieni i tuoi **repository di codice** (con Git sotto, come motore).
2. Come servizio è una **piattaforma pronta (PaaS)**, della famiglia **…aaS**, e si paga in **Freemium** (base gratis, extra a pagamento).
3. Il progetto cresce per **versioni** (1.2.3): major, minor, patch — e una versione pubblicata non si tocca più.

Per vedere tante situazioni diverse con i grafi, guarda la scheda "Versioning — 11 casi d'uso".

9. Glossario

Parola	Vuol dire
GitHub	il sito-servizio (azienda di Microsoft) che ospita i progetti online
Git	lo strumento (gratis) che tiene la storia di un progetto — il "motore" sotto GitHub
repository	la cartella-progetto con dentro i file e tutta la loro storia
commit	un salvataggio (una fotografia) del progetto con un messaggio
branch	una strada di lavoro parallela, per provare senza rovinare il main
merge	unire un ramo con quello principale (main)
PaaS	"piattaforma come servizio": una piattaforma online già pronta (es. GitHub)
IaaS / SaaS	gli altri servizi …aaS: infrastruttura (server nudi) / software pronto nel browser
Freemium	base gratis, funzioni avanzate a pagamento
versioning	dare un numero (v1.2.3) alle versioni: major . minor . patch
release	una versione pubblicata e "congelata"; non si riusa mai un numero già dato

Changelog

1. **v1.0 (29/10/2026)**: prima versione, dalle fonti della lezione.